

DevOps Handbook

1 DevOps Handbook — Master Table of Contents

- [1.1 How to read this handbook](#)
- [1.2 Part I — Introduction & Mindset \(Pages 1–60\)](#)
- [1.3 Part II — Linux & Shell Mastery \(Pages 61–140\)](#)
- [1.4 Part III — Git & Collaboration \(Pages 141–200\)](#)
- [1.5 Part IV — Networking for DevOps \(Pages 201–280\)](#)
- [1.6 Part V — Scripting & Programming \(Pages 281–360\)](#)
- [1.7 Part VI — Infrastructure as Code \(Pages 361–480\)](#)
- [1.8 Part VII — Containers \(Pages 481–580\)](#)
- [1.9 Part VIII — Kubernetes \(Pages 581–720\)](#)
- [1.10 Part IX — CI/CD \(Pages 721–840\)](#)
- [1.11 Part X — Observability & SRE \(Pages 841–960\)](#)
- [1.12 Part XI — Security \(DevSecOps\) \(Pages 961–1080\)](#)
- [1.13 Part XII — Advanced Topics \(Pages 1081–1200\)](#)
- [1.14 Expansion status](#)

2 Chapter 1: What Is DevOps? History and Culture

- [2.1 1.1 The problem DevOps solves](#)
- [2.2 1.2 A working definition](#)
- [2.3 1.3 Brief history](#)
- [2.4 1.4 Culture before tools](#)
- [2.5 1.5 DevOps practices \(overview\)](#)
- [2.6 1.6 Organizational models](#)
 - [2.6.1 Functional silos \(anti-pattern for DevOps\)](#)
 - [2.6.2 Cross-functional product teams \(recommended\)](#)
 - [2.6.3 DevOps enablement / platform team](#)
- [2.7 1.7 What DevOps is NOT](#)
- [2.8 1.8 Your learning path in this handbook](#)
- [2.9 1.9 Chapter summary](#)
- [2.10 Lab 1.1 — Reflect on your current workflow](#)
- [2.11 Review questions](#)

3 Chapter 2: CALMS, Three Ways, and Team Topologies

- [3.1 2.1 The CALMS framework](#)
 - [3.1.1 Culture \(C\)](#)
 - [3.1.2 Automation \(A\)](#)
 - [3.1.3 Lean \(L\)](#)
 - [3.1.4 Measurement \(M\)](#)
 - [3.1.5 Sharing \(S\)](#)
- [3.2 2.2 The Three Ways \(Gene Kim\)](#)
 - [3.2.1 First Way: Flow \(left to right\)](#)
 - [3.2.2 Second Way: Feedback \(right to left\)](#)
 - [3.2.3 Third Way: Continuous learning and experimentation](#)
- [3.3 2.3 Team Topologies \(Skelton & Pais\)](#)
 - [3.3.1 Interaction modes](#)
- [3.4 2.4 Conway's Law and reverse Conway maneuver](#)
- [3.5 2.5 Maturity model \(informal\)](#)
- [3.6 2.6 Chapter summary](#)
- [3.7 Lab 2.1 — CALMS assessment](#)
- [3.8 Review questions](#)

4 Chapter 3: DevOps vs SRE vs Platform Engineering

- [4.1 3.1 Three related disciplines](#)
- [4.2 3.2 Site Reliability Engineering \(SRE\)](#)
 - [4.2.1 Core SRE principles](#)
 - [4.2.2 SRE team responsibilities](#)

[4.2.3 Error budget example](#)
[4.3 3.3 Platform Engineering](#)
[4.3.1 What platform teams build](#)
[4.3.2 Platform vs traditional ops](#)
[4.4 3.4 How they fit together](#)
[4.5 3.5 Job titles in the market](#)
[4.6 3.6 Chapter summary](#)
[4.7 Lab 3.1](#)

5 Chapter 5: Linux Fundamentals — Filesystem, Users, Permissions

[5.1 5.1 Why Linux dominates DevOps](#)
[5.2 5.2 Filesystem hierarchy \(FHS\)](#)
[5.3 5.3 Essential navigation commands](#)
[5.4 5.4 File operations](#)
[5.4.1 Finding files](#)
[5.5 5.5 Users, groups, and permissions](#)
[5.5.1 Users and groups](#)
[5.5.2 Permission model](#)
[5.5.3 Special permissions](#)
[5.6 5.6 Package management \(Debian/Ubuntu\)](#)
[5.7 5.7 Text processing \(daily DevOps tools\)](#)
[5.8 5.8 Environment and shell](#)
[5.9 5.9 SSH — remote access](#)
[5.10 5.10 Chapter summary](#)
[5.11 Lab 5.1 — Linux sandbox](#)
[5.12 Review questions](#)

6 Chapter 6: Processes, systemd, and Service Management

[6.1 6.1 Understanding Linux processes](#)
[6.1.1 Process states](#)
[6.1.2 Signals](#)
[6.2 6.2 systemd — modern service manager](#)
[6.2.1 Unit files](#)
[6.3 6.3 Cron and timers](#)
[6.3.1 cron — scheduled tasks](#)
[6.3.2 systemd timers \(preferred on modern systems\)](#)
[6.4 6.4 Resource limits and troubleshooting](#)
[6.4.1 Common production issues](#)
[6.5 6.5 Chapter summary](#)
[6.6 Lab 6.1](#)

7 Chapter 9: Git Internals — Objects, Refs, and the DAG

[7.1 9.1 Git is a content-addressable filesystem](#)
[7.2 9.2 How a commit is stored](#)
[7.3 9.3 Essential Git commands \(beyond basics\)](#)
[7.3.1 Undoing mistakes](#)
[7.4 9.4 Branches and merging](#)
[7.4.1 Merge vs rebase](#)
[7.5 9.5 Remote repositories](#)
[7.5.1 Tracking branches](#)
[7.6 9.6 .gitignore and hooks](#)
[7.6.1 .gitignore](#)
[7.6.2 Git hooks \(`.git/hooks/` \)](#)
[7.7 9.7 Chapter summary](#)
[7.8 Lab 9.1](#)

8 Chapter 20: IaC Principles — State, Idempotency, and Drift

[8.1 20.1 What is Infrastructure as Code?](#)
[8.2 20.2 Declarative vs imperative](#)
[8.3 20.3 Idempotency](#)
[8.4 20.4 State management](#)

[8.4.1 State best practices](#)
[8.5 20.5 Configuration drift](#)
[8.6 20.6 IaC workflow in teams](#)
[8.6.1 Directory structure \(example\)](#)
[8.7 20.7 Testing IaC](#)
[8.8 20.8 Chapter summary](#)
[8.9 Lab 20.1](#)

[9 Chapter 21: Terraform — HCL, Providers, and Modules](#)

[9.1 21.1 Terraform overview](#)
[9.2 21.2 Basic configuration](#)
[9.3 21.3 Resources, data sources, and dependencies](#)
[9.3.1 Data sources — read existing infrastructure](#)
[9.3.2 Implicit vs explicit dependencies](#)
[9.4 21.4 Modules — reusable infrastructure](#)
[9.5 21.5 Lifecycle meta-arguments](#)
[9.6 21.6 Workspaces and environments](#)
[9.7 21.7 CI/CD integration](#)
[9.8 21.8 Chapter summary](#)
[9.9 Lab 21.1](#)

[10 Chapter 26: Docker — Images, Dockerfile, and Multi-stage Builds](#)

[10.1 26.1 Containers in one paragraph](#)
[10.2 26.2 Core concepts](#)
[10.3 26.3 Essential commands](#)
[10.4 26.4 Dockerfile best practices](#)
[10.4.1 Best practices checklist](#)
[10.5 26.5 Docker Compose for local development](#)
[10.6 26.6 Security basics](#)
[10.7 26.7 Chapter summary](#)
[10.8 Lab 26.1](#)

[11 Chapter 37: CI/CD Concepts and Pipeline Design](#)

[11.1 37.1 Continuous Integration \(CI\)](#)
[11.2 37.2 Continuous Delivery vs Continuous Deployment](#)
[11.3 37.3 Pipeline stages](#)
[11.4 37.4 GitHub Actions example](#)
[11.5 37.5 Branching and trunk-based development](#)
[11.6 37.6 Deployment strategies](#)
[11.7 37.7 Pipeline secrets and artifacts](#)
[11.8 37.8 DORA metrics in your pipeline](#)
[11.9 37.9 Chapter summary](#)
[11.10 Lab 37.1](#)

1 DevOps Handbook — Master Table of Contents

Target length: ~1,200 pages (~480,000 words)
Audience: Beginner → Advanced
Format: Each numbered section ≈ one “page” of ~400 words in the final edition

1.1 How to read this handbook

Work through **Parts I-XII** in order. Labs (marked) add hands-on time beyond page counts. Cross-reference the [AWS Handbook](#) for cloud-specific depth.

1.2 Part I — Introduction & Mindset (Pages 1-60)

Ch	Title	Pages
1	What Is DevOps? History and Culture	1-15
2	CALMS, Three Ways, and Team Topologies	16-30
3	DevOps vs SRE vs Platform Engineering	31-42
4	Career Paths, Certifications, and Learning Strategy	43-60

Written: Chapters 1-3, 5-6, 9, 20-21, 26, 37 (11 chapters + full TOC)

1.3 Part II — Linux & Shell Mastery (Pages 61-140)

Ch	Title	Pages
5	Linux Fundamentals: Filesystem, Users, Permissions	61-85
6	Processes, systemd, and Service Management	86-105
7	Networking on Linux: ip, ss, curl, dig	106-120
8	Bash Scripting for Automation	121-140

Written: [Chapter 5](#), [Chapter 6](#)

1.4 Part III — Git & Collaboration (Pages 141-200)

Ch	Title	Pages
9	Git Internals: Objects, Refs, and the DAG	141-158
10	Branching Strategies and Code Review	159-175
11	GitHub/GitLab Workflows: PRs, Issues, Actions intro	176-200

1.5 Part IV — Networking for DevOps (Pages 201-280)

Ch	Title	Pages
12	OSI Model, TCP/IP, DNS Deep Dive	201-225
13	HTTP/HTTPS, TLS, and Certificates	226-245
14	Load Balancing, Proxies, and CDN Concepts	246-265
15	Cloud Networking Patterns (VPC overview)	266-280

1.6 Part V — Scripting & Programming (Pages 281-360)

Ch	Title	Pages
16	Python for DevOps: boto3, requests, pathlib	281-305
17	Go for CLI Tools and Operators	306-325
18	JSON/YAML, Jinja2, and Templating	326-345
19	Testing Automation Scripts	346-360

1.7 Part VI — Infrastructure as Code (Pages 361-480)

Ch	Title	Pages
20	IaC Principles: State, Idempotency, Drift	361-380
21	Terraform: HCL, Providers, Modules	381-410
22	Terraform: Workspaces, Backends, CI Integration	411-430
23	Ansible: Playbooks, Roles, Inventories	431-450
24	Packer & Golden Images	451-465
25	Policy as Code: OPA, Checkov, Sentinel	466-480

1.8 Part VII — Containers (Pages 481-580)

Ch	Title	Pages
26	Docker: Images, Dockerfile, Multi-stage Builds	481-505
27	Container Networking, Volumes, and Security	506-525
28	Docker Compose and Local Dev Environments	526-540
29	Container Registries and Image Signing	541-560
30	Rootless Containers and Alternatives (Podman)	561-580

1.9 Part VIII — Kubernetes (Pages 581-720)

Ch	Title	Pages
31	K8s Architecture: Control Plane & Node Components	581–605
32	Workloads: Pod, Deployment, StatefulSet, DaemonSet	606–630
33	Services, Ingress, and Network Policies	631–655
34	ConfigMaps, Secrets, and Storage (PV/PVC)	656–675
35	Helm, Kustomize, and GitOps (Argo CD)	676–700
36	RBAC, Pod Security, and Hardening	701–720

1.10 Part IX — CI/CD (Pages 721–840)

Ch	Title	Pages
37	CI/CD Concepts and Pipeline Design	721–745
38	GitHub Actions: Workflows, Secrets, Runners	746–770
39	Jenkins: Pipelines as Code (Declarative)	771–790
40	GitLab CI and Multi-platform Comparison	791–810
41	Deployment Strategies: Blue/Green, Canary, Feature Flags	811–825
42	DORA Metrics and Continuous Improvement	826–840

1.11 Part X — Observability & SRE (Pages 841–960)

Ch	Title	Pages
43	Monitoring Fundamentals: Metrics, Logs, Traces	841–865
44	Prometheus & Grafana Stack	866–890
45	ELK/EFK and Log Aggregation	891–910
46	OpenTelemetry and Distributed Tracing	911–930
47	SLOs, Error Budgets, and Incident Response	931–945
48	Chaos Engineering and Game Days	946–960

1.12 Part XI — Security (DevSecOps) (Pages 961–1080)

Ch	Title	Pages
49	Shift-Left Security in the Pipeline	961–980
50	SAST, DAST, SCA, and Container Scanning	981–1000
51	Secrets Management: Vault, SOPS, External Secrets	1001–1020

52	Supply Chain: SBOM, Sigstore, SLSA	1021-1040
53	Runtime Security: Falco, OPA Gatekeeper	1041-1060
54	Compliance Automation and Audit Trails	1061-1080

1.13 Part XII — Advanced Topics (Pages 1081-1200)

Ch	Title	Pages
55	Platform Engineering & Internal Developer Platforms	1081-1100
56	Multi-Cloud and Hybrid Patterns	1101-1115
57	FinOps and Cost-Aware Engineering	1116-1130
58	MLOps Overview for DevOps Engineers	1131-1145
59	Capstone: End-to-End Production System Design	1146-1180
60	Appendix: Command Cheatsheets, Glossary, Index	1181-1200

1.14 Expansion status

Part	Status
I	Ch 1-3, 5-6, 9, 20-21, 26, 37 written (~120 pages of content)

To request a chapter: *“Expand DevOps Handbook Chapter 21 (Terraform modules)”*

2 Chapter 1: What Is DevOps? History and Culture

DevOps Handbook — Part I, Pages 1–15

2.1 1.1 The problem DevOps solves

Before DevOps, software organizations often split into two camps:

- **Development** — wrote features quickly, measured by release velocity.
- **Operations** — kept systems stable, measured by uptime and incident count.

These goals frequently conflicted. Developers threw code “over the wall”; operators resisted change because every deployment risked an outage. Releases were rare, painful, and manual. When something broke in production, blame flowed in both directions.

DevOps is not a single tool or job title. It is a **cultural and technical movement** that unifies development and operations around shared goals: deliver value to users **fast** and **reliably**.

2.2 1.2 A working definition

***DevOps** is the practice of aligning people, processes, and technology so that software can be built, tested, released, and operated continuously—with feedback loops that shorten the time from idea to production.*

Key outcomes:

Outcome	What it means
Shorter lead time	Hours or days from commit to production, not months
Higher deployment frequency	Many small releases instead of rare big-bang releases
Lower change failure rate	Most changes succeed without rollback
Faster recovery	When failures happen, restore service quickly

These four metrics are known as **DORA metrics** (DevOps Research and Assessment) and are the standard way to measure DevOps maturity.

2.3 1.3 Brief history

Year	Milestone
2007–2009	Flickr, Etsy, and others share “10+ deploys per day” stories
2009	First DevOpsDays conference in Ghent, Belgium (Patrick Debois)
2010	“The Phoenix Project” novel popularizes DevOps narrative
2013	“The DevOps Handbook” (Gene Kim et al.) codifies practices
2016+	Cloud-native, containers, and Kubernetes accelerate adoption
2020s	Platform engineering, GitOps, and AI-assisted ops emerge

The term **DevOps** blends **Development** + **Operations**. It intentionally breaks down silos rather than creating a third silo called “DevOps team” that sits between dev and ops.

2.4 1.4 Culture before tools

A common mistake is buying Jenkins, Kubernetes, or Terraform and calling it “DevOps.” Tools enable DevOps; they do not create it.

Cultural pillars:

1. **Shared ownership** — The team that builds a service also runs it (“you build it, you run it”).
 2. **Blameless postmortems** — Focus on systems and processes, not individuals, after incidents.
 3. **Automation of toil** — Repetitive manual work is eliminated or codified.
 4. **Continuous learning** — Experimentation, feedback, and improvement are expected.
 5. **Customer focus** — Every decision ties back to user value.
-

2.5 1.5 DevOps practices (overview)

You will study each in depth later in this handbook:

Practice	Summary
Version control	Everything in Git: code, config, IaC, docs
CI/CD	Automated build, test, deploy pipelines
Infrastructure as Code	Reproducible environments (Terraform, Ansible)
Monitoring & observability	Know what production is doing in real time
Microservices & containers	Smaller deployable units (optional but common)
Security integration (DevSecOps)	Security checks in the pipeline, not at the end

2.6 1.6 Organizational models

2.6.1 Functional silos (anti-pattern for DevOps)

Separate dev, QA, ops, and security teams with handoffs.

2.6.2 Cross-functional product teams (recommended)

One team owns a product or service end-to-end: code, infra, on-call, roadmap.

2.6.3 DevOps enablement / platform team

A **platform team** builds internal tools (CI templates, K8s clusters, golden paths) so product teams move faster. This is **not** a ticket-taking ops team—it is a **product for developers**.

2.7 1.7 What DevOps is NOT

- **Not “NoOps”** — Operations expertise is more important at scale, not less.
- **Not only for startups** — Enterprises adopt DevOps for compliance and speed.

- **Not replacing developers with operators** — It merges responsibilities skillfully.
- **Not a certification checkbox** — Culture and practice matter more than badges.

2.8 1.8 Your learning path in this handbook

```
graph LR; Foundations --> Automation --> Containers --> CI_CD[CI/CD] --> Observability --> Security --> Advanced; Linux_Git_Networking[Linux, Git, Networking] --- Platform_SRE[Platform & SRE];
```

Invest 70% of study time in **hands-on labs**. Reading without doing builds false confidence.

2.9 1.9 Chapter summary

- DevOps unifies dev and ops around **speed** and **reliability**.
- **Culture** (shared ownership, blameless learning) precedes tools.
- Measure progress with **DORA metrics**.
- Cross-functional teams and platform engineering are modern expressions of DevOps.

2.10 Lab 1.1 — Reflect on your current workflow

1. Map your (or a sample) team's path from code commit to production.
2. Count manual steps and handoffs.
3. Identify one bottleneck and one automation opportunity.
4. Write a one-paragraph "DevOps vision" for that team.

2.11 Review questions

1. Name the four DORA metrics.
2. Why is “DevOps team” as a silo problematic?
3. What year and event is credited with coining “DevOps”?
4. Give one cultural and one technical practice of DevOps.

Next: Chapter 2 — CALMS, Three Ways, and Team Topologies

3 Chapter 2: CALMS, Three Ways, and Team Topologies

DevOps Handbook — Part I, Pages 16–30

3.1 2.1 The CALMS framework

CALMS is a mnemonic for assessing DevOps maturity across five dimensions:

Letter	Dimension	Description
C	Culture	Collaboration, trust, shared goals
A	Automation	Eliminate manual, error-prone work
L	Lean	Reduce waste, small batches, flow
M	Measurement	Data-driven decisions, DORA metrics
S	Sharing	Knowledge, tools, failures, successes

3.1.1 Culture (C)

Healthy DevOps culture means developers understand production constraints and operators understand business priorities. Psychological safety allows people to report problems early.

3.1.2 Automation (A)

Automate builds, tests, deployments, infrastructure provisioning, and compliance checks. Manual runbooks become code. **Toil**—repetitive operational work that scales linearly with service growth—should be targeted for elimination.

3.1.3 Lean (L)

Inspired by Toyota Production System: optimize **flow**, limit work in progress (WIP), and deliver **small batches**. Large releases hide defects; small releases surface them quickly.

3.1.4 Measurement (M)

You cannot improve what you do not measure. Track deployment frequency, lead time, MTTR, change failure rate, plus business KPIs (revenue, user satisfaction).

3.1.5 Sharing (S)

Internal conferences, shared Slack channels, communal dashboards, and open postmortems spread expertise. Tools and patterns are reused, not reinvented per team.

3.2 2.2 The Three Ways (Gene Kim)

From *The Phoenix Project* and *The DevOps Handbook*, the **Three Ways** describe the principles of DevOps flow:

3.2.1 First Way: Flow (left to right)

Accelerate delivery from **development** → **operations** → **customer**.

- Make work visible (Kanban boards, pipeline status).
- Reduce batch sizes.
- Build quality in (automated tests at every stage).
- Never pass defects downstream.

Example: A commit triggers CI; only green builds reach staging; only approved artifacts reach production.

3.2.2 Second Way: Feedback (right to left)

Amplify feedback from **production → development**.

- Monitor user experience and system health.
- Swarm on problems; shorten feedback loops.
- Developers get production access (with guardrails).

Example: Error rate spike in Grafana pages the on-call engineer; linked traces show the bad deploy; rollback is automated.

3.2.3 Third Way: Continuous learning and experimentation

Create a culture of **improvement** and **risk-taking**.

- Blameless postmortems.
- Reserve time for improvement (20% ops time, hack days).
- Rehearse failures (game days, chaos engineering).

3.3 2.3 Team Topologies (Skelton & Pais)

Modern DevOps organizations structure teams intentionally:

Team type	Purpose
Stream-aligned	Delivers product features; primary value stream
Platform	Provides self-service infra (K8s, CI, databases)
Enabling	Coaches stream teams on new tech (temporary)
Complicated-subsystem	Owns deep specialty (e.g., video encoding, ML)

3.3.1 Interaction modes

- **Collaboration** — Two teams work closely for discovery.
- **X-as-a-Service** — Platform provides API/portal; stream teams consume.
- **Facilitating** — Enabling team mentors others.

Anti-pattern: One ops team serves 50 dev teams via tickets.

Better: Platform team offers paved roads; stream teams deploy autonomously.

3.4 2.4 Conway's Law and reverse Conway maneuver

“Organizations design systems that mirror their communication structure.” — Melvin Conway

If dev and ops are separate departments, you will get separate deployment artifacts and fragile handoffs.

Reverse Conway maneuver: Structure teams the way you want the architecture to look. Want autonomous microservices? Give teams end-to-end ownership of each service.

3.5 2.5 Maturity model (informal)

Level	Characteristics
1 — Initial	Manual deploys, silos, hero culture
2 — Managed	Some CI, version control, basic monitoring
3 — Defined	IaC, automated testing, standardized pipelines
4 — Measured	SLOs, DORA tracking, blameless postmortems
5 — Optimizing	Continuous experimentation, platform self-service

Use this for self-assessment, not as a rigid certification ladder.

3.6 2.6 Chapter summary

- **CALMS** evaluates Culture, Automation, Lean, Measurement, Sharing.
 - The **Three Ways** optimize flow, feedback, and learning.
 - **Team Topologies** align org structure with delivery goals.
 - Apply **Conway's Law** consciously when designing teams.
-

3.7 Lab 2.1 — CALMS assessment

Score your team 1–5 on each CALMS letter. Pick the lowest score and draft one actionable improvement per dimension.

3.8 Review questions

1. What does the “L” in CALMS stand for, and what Lean concept supports it?
 2. Describe the First Way in one sentence.
 3. Name the four team types in Team Topologies.
 4. What is a reverse Conway maneuver?
-

Next: Part II — [Chapter 5: Linux Fundamentals](#)

4 Chapter 3: DevOps vs SRE vs Platform Engineering

DevOps Handbook — Part I, Pages 31–42

4.1 3.1 Three related disciplines

DevOps, **Site Reliability Engineering (SRE)**, and **Platform Engineering** are often confused because they share goals: reliable systems, fast delivery, and reduced toil. They are complementary, not competing.

Discipline	Origin	Primary focus
DevOps	Movement (2009+)	Culture + practices bridging dev and ops
SRE	Google (2003+, public ~2016)	Engineering approach to operations at scale
Platform Engineering	Industry evolution (2020s)	Internal platforms that accelerate product teams

4.2 3.2 Site Reliability Engineering (SRE)

Google introduced **SRE** as “what happens when you ask a software engineer to design an operations function.”

4.2.1 Core SRE principles

1. **Embrace risk** — 100% uptime is impossible and expensive; use **error budgets** to balance reliability and velocity.
2. **Service level objectives (SLOs)** — Define measurable targets (e.g., 99.9% availability).
3. **Eliminate toil** — Manual, repetitive work that scales with traffic must be automated.
4. **Monitoring** — Systems must be observable; alerts fire on symptoms users feel.
5. **Release engineering** — Safe, gradual rollouts with fast rollback.
6. **Simplicity** — Prefer boring, well-understood technology.

4.2.2 SRE team responsibilities

- Define and track SLOs/SLIs
- On-call rotation and incident response
- Capacity planning
- Performance and reliability testing
- Building automation to replace toil

4.2.3 Error budget example

If SLO = 99.9% monthly availability, you have ~43 minutes of downtime budget. When budget is exhausted, **freeze feature releases** and focus on reliability until budget recovers.

4.3 3.3 Platform Engineering

Platform engineering treats the internal developer platform as a **product** whose customers are other

engineers.

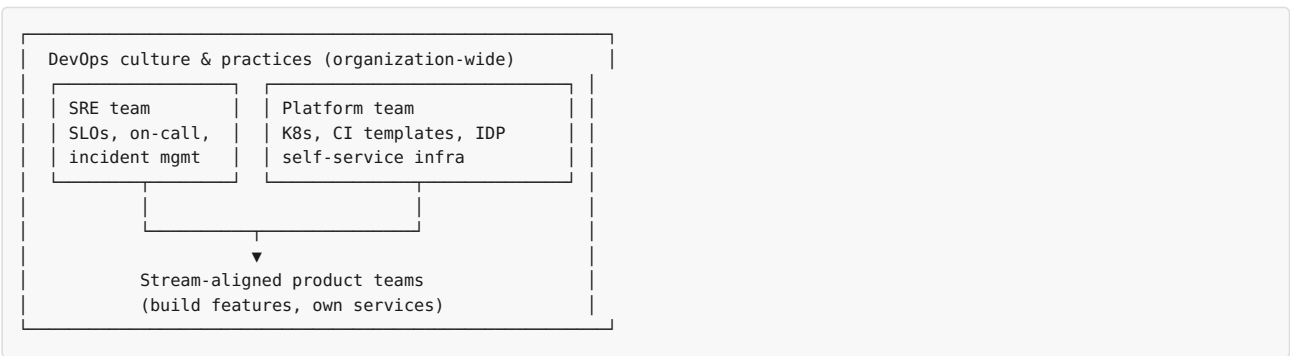
4.3.1 What platform teams build

- Kubernetes clusters with guardrails
- CI/CD templates and golden pipelines
- Self-service databases, caches, and queues
- Developer portals (Backstage) with docs and service catalogs
- Standardized observability (dashboards, tracing, logging)

4.3.2 Platform vs traditional ops

Traditional ops	Platform engineering
Ticket queue for infra requests	Self-service APIs and portals
Snowflake servers	Standardized golden paths
“Call us to deploy”	Teams deploy autonomously with paved roads

4.4 3.4 How they fit together



- **DevOps** sets the cultural north star.
- **SRE** ensures reliability is measurable and engineered.
- **Platform** removes friction so product teams ship faster.

4.5 3.5 Job titles in the market

Titles vary widely. Focus on **responsibilities**, not labels:

Title	Often means
DevOps Engineer	CI/CD, IaC, cloud infra, automation
SRE	SLOs, on-call, reliability tooling, toil reduction
Platform Engineer	Internal platform, K8s, developer experience
Cloud Engineer	AWS/Azure/GCP architecture and implementation
Infrastructure Engineer	Servers, networking, IaC

Many roles blend all three disciplines.

4.6 3.6 Chapter summary

- **SRE** engineers reliability with SLOs, error budgets, and toil automation.
 - **Platform engineering** provides self-service paved roads for developers.
 - **DevOps** is the umbrella culture; SRE and platform are specialized implementations.
-

4.7 Lab 3.1

Draft SLOs for a sample API: pick one SLI (latency or availability), set a target, and calculate the monthly error budget in minutes.

Next: [Chapter 6 — Processes and systemd](#)

5 Chapter 5: Linux Fundamentals — Filesystem, Users, Permissions

DevOps Handbook — Part II, Pages 61–85

5.1 5.1 Why Linux dominates DevOps

Most servers, containers, and cloud instances run **Linux**. As a DevOps engineer you will SSH into machines, debug production issues, write shell scripts, and configure services daily. Windows matters too (Active Directory, .NET shops), but Linux fluency is non-negotiable.

Common distributions:

Distro	Typical use
Ubuntu	Cloud default, tutorials, desktops
Debian	Stable base for Ubuntu and containers
RHEL / Rocky / Alma	Enterprise, regulated industries
Amazon Linux	AWS-optimized

5.2 5.2 Filesystem hierarchy (FHS)

Path	Purpose
/	Root of entire tree
/home	User home directories (/home/alice)
/root	root user’s home
/etc	Configuration files
/var	Variable data: logs (/var/log), caches
/tmp	Temporary files (often cleared on reboot)
/usr	User programs and libraries
/bin , /sbin	Essential binaries
/opt	Optional third-party software
/proc , /sys	Virtual filesystems for kernel info

Everything is a **file** in Unix—including devices and sockets.

5.3 5.3 Essential navigation commands

```
pwd           # Print working directory
ls -lah       # List files (human-readable, including hidden)
cd /etc/nginx # Change directory
cd ..         # Parent directory
cd ~          # Home directory
```

```
tree -L 2 /etc      # Tree view (install: apt install tree)
```

```
cp source dest           # Copy
cp -r dir1 dir2          # Copy directory recursively
mv old new               # Move or rename
rm file                  # Delete (careful!)
rm -rf directory         # Delete directory recursively (very careful!)
mkdir -p path/to/dir     # Create nested directories
touch file.txt           # Create empty file or update timestamp
cat file                 # Print file contents
less /var/log/syslog     # Paginated view (q to quit)
head -n 20 file          # First 20 lines
tail -f /var/log/app.log # Follow log in real time
```

```
find /var/log -name "*.log" -mtime -7 # Logs modified in 7 days
find / -type f -size +100M 2>/dev/null # Files larger than 100MB
locate nginx.conf # Fast index search (updatedb)
```

```
id                # Current user UID, GID, groups
whoami
cat /etc/passwd   # User accounts
cat /etc/group     # Groups
sudo command      # Run as root (if permitted)
su - username     # Switch user
```

Each file has three permission sets for **user (u)**, **group (g)**, **others (o)**:

```
-rwxr-xr-- 1 alice developers 4096 Jan 15 10:00 deploy.sh
|||||
|||  others: read
||   group: read, execute
|    user: read, write, execute
```

```
chmod 755 script.sh      # rwXr-Xr-X
chmod u+x script.sh      # Add execute for user
chown alice:developers file
chgrp developers file
```

5.5.3 Special permissions

- **setuid** — Run as file owner (e.g., `passwd`)
- **setgid** — Run as group; dirs inherit group
- **sticky bit** — On `/tmp` , only owner can delete own files

5.6 5.6 Package management (Debian/Ubuntu)

```
sudo apt update           # Refresh package index
sudo apt upgrade -y       # Upgrade installed packages
sudo apt install nginx -y # Install package
apt search nginx          # Search
apt show nginx            # Package info
sudo apt remove nginx     # Remove
dpkg -l | grep nginx      # List installed
```

RHEL/Rocky equivalent: `dnf install` , `dnf update` .

5.7 5.7 Text processing (daily DevOps tools)

```
grep -r "ERROR" /var/log/app/ # Search recursively
grep -E "timeout|refused" log.txt # Extended regex
awk '{print $1}' access.log     # Column extraction
sed 's/old/new/g' file.txt      # Stream editor
sort file | uniq -c | sort -rn | head # Count unique lines
wc -l file                     # Line count
```

Pipes (|) chain commands: stdout of one → stdin of next.

5.8 5.8 Environment and shell

```
echo $PATH                # Executable search path
export MY_VAR=hello       # Set environment variable
env                       # All environment variables
which python3             # Path to executable
alias ll='ls -lah'        # Command shortcut
```

Shell config files: `~/.bashrc` , `~/.profile` (sourced on login).

5.9 5.9 SSH — remote access

```
ssh user@hostname
ssh -i ~/.ssh/mykey.pem ec2-user@1.2.3.4
scp file user@host:/remote/path
rsync -avz ./local/ user@host:/remote/ # Efficient sync
```

`~/.ssh/config` simplifies connections:

```
Host prod-web
  HostName 10.0.1.50
  User ubuntu
  IdentityFile ~/.ssh/prod.pem
```

Then: `ssh prod-web`

5.10 Chapter summary

- Linux FHS organizes `/etc`, `/var`, `/home`, and more.
 - Master `ls`, `cd`, `cp`, `mv`, `rm`, `find`, `grep`, `tail -f`.
 - Permissions: `user/group/other` + `chmod` / `chown`.
 - Use `apt` / `dnf` for packages; SSH for remote ops.
-

5.11 Lab 5.1 — Linux sandbox

1. Create user `devops-lab` with home directory.
 2. Create `~/scripts/deploy.sh` with `#!/bin/bash` and `echo "Deploying..."`.
 3. Set permissions to `755`; run it.
 4. Write a one-liner to count ERROR lines in a sample log file.
 5. Configure SSH key auth to a local VM or cloud instance.
-

5.12 Review questions

1. What directory holds system configuration files?
 2. What does permission `644` mean in symbolic notation?
 3. How do you follow a log file in real time?
 4. What is the difference between `cp` and `mv`?
-

Continue: Chapter 6 — Processes, systemd, and Service Management (outline in TOC)

6 Chapter 6: Processes, systemd, and Service Management

DevOps Handbook — Part II, Pages 86–105

6.1 6.1 Understanding Linux processes

A **process** is a running instance of a program. Each has a **PID** (Process ID), owner, state, and resource usage.

```
ps aux          # All processes (BSD style)
ps -ef         # All processes (POSIX style)
top            # Interactive process viewer
htop          # Enhanced top (install separately)
pgrep nginx    # Find PIDs by name
pkill -HUP nginx # Send signal to processes by name
```

6.1.1 Process states

State	Code	Meaning
Running	R	Executing or runnable
Sleeping	S	Waiting for event
Zombie	Z	Terminated but not reaped by parent
Stopped	T	Paused (e.g., Ctrl+Z)

6.1.2 Signals

```
kill -15 <pid> # SIGTERM – graceful shutdown (default)
kill -9 <pid>  # SIGKILL – force kill (last resort)
kill -HUP <pid> # SIGHUP – reload config (many daemons)
```

6.2 6.2 systemd — modern service manager

Most Linux distros use **systemd** to boot the system and manage services (units).

```
systemctl status nginx
systemctl start nginx
systemctl stop nginx
systemctl restart nginx
systemctl reload nginx # Reload config without full restart
systemctl enable nginx # Start on boot
systemctl disable nginx
systemctl list-units --type=service --state=running
journalctl -u nginx -f # Follow service logs
journalctl -u nginx --since "1 hour ago"
```

6.2.1 Unit files

Service definitions live in `/etc/systemd/system/` or `/lib/systemd/system/`.

Example `/etc/systemd/system/myapp.service`:

```
[Unit]
```

```

Description=My Application
After=network.target

[Service]
Type=simple
User=appuser
WorkingDirectory=/opt/myapp
ExecStart=/opt/myapp/bin/server
Restart=on-failure
RestartSec=5
Environment=PORT=8080

[Install]
WantedBy=multi-user.target

```

After creating or editing a unit:

```

sudo systemctl daemon-reload
sudo systemctl enable --now myapp

```

6.3 Cron and timers

6.3.1 cron — scheduled tasks

```

crontab -e      # Edit user crontab
crontab -l      # List crontab

```

Cron format: minute hour day month weekday command

```

# Every day at 2:30 AM – backup database
30 2 * * * /opt/scripts/backup.sh >> /var/log/backup.log 2>&1

```

System-wide jobs: /etc/cron.d/, /etc/cron.daily/, etc.

6.3.2 systemd timers (preferred on modern systems)

Timers are more flexible and integrate with journald logging.

```

# /etc/systemd/system/backup.timer
[Unit]
Description=Daily backup

[Timer]
OnCalendar=*-*-* 02:30:00
Persistent=true

[Install]
WantedBy=timers.target

```

```

systemctl enable --now backup.timer
systemctl list-timers

```

6.4 Resource limits and troubleshooting

```

# CPU and memory per process
ps -o pid,user,%cpu,%mem,cmd -p <pid>

# Open files by process
lsof -p <pid>
ss -tlnp      # Listening TCP ports with process

```

```
df -h          # Disk usage
free -h        # Memory usage
dmesg | tail    # Kernel messages
```

6.4.1 Common production issues

Symptom	Likely cause	Check
Service won't start	Bad config, missing binary	<code>journalctl -u service -n 50</code>
Port in use	Another process bound	<code>ss -tlnp \ grep :80</code>
Disk full	Logs, temp files	<code>df -h, du -sh /var/log/*</code>
OOM killed	Memory exhaustion	<code>dmesg \ grep -i oom</code>

6.5 Chapter summary

- Manage processes with `ps`, `kill`, and signals.
- Use **systemctl** and **journalctl** for services on systemd systems.
- Schedule jobs with cron or systemd timers.
- Check logs first when debugging service failures.

6.6 Lab 6.1

1. Create a simple bash script that writes a timestamp to a log every minute.
2. Run it as a systemd service + timer.
3. Practice `systemctl status`, `journalctl -f`, and graceful stop/start.

Next: [Chapter 9 — Git Internals](#)

7 Chapter 9: Git Internals — Objects, Refs, and the DAG

DevOps Handbook — Part III, Pages 141–158

7.1 9.1 Git is a content-addressable filesystem

At its core, Git stores **snapshots** of your project, not just diffs. Every piece of content is identified by a **SHA-1 hash** (Git is migrating to SHA-256).

Four object types:

Type	Contains
blob	File content
tree	Directory listing (names → blobs/trees)
commit	Snapshot + metadata (author, message, parent)
tag	Named pointer to a commit (annotated tags)

7.2 9.2 How a commit is stored

When you `git add` and `git commit`:

1. Git compresses file content into **blobs**.
2. Directory structure becomes **trees**.
3. A **commit** object points to the root tree and parent commit(s).
4. Branch name (e.g., `main`) is a **ref** — a pointer to the latest commit.

```
main → commit C3 → tree → blob (README.md)
      ↓
      commit C2 → ...
      ↓
      commit C1 (root)
```

This forms a **DAG** (directed acyclic graph). Merge commits have two parents.

7.3 9.3 Essential Git commands (beyond basics)

```
git log --oneline --graph --all
git show <commit>           # Commit details + diff
git diff HEAD~1             # Compare with previous commit
git stash push -m "wip"
git stash pop
git cherry-pick <commit>
git rebase main              # Replay commits on top of main
git reflog                  # History of HEAD movements (recovery!)
```

7.3.1 Undoing mistakes

```
git restore file.txt        # Discard working tree changes
```

```
git restore --staged file.txt # Unstage
git reset --soft HEAD~1      # Undo commit, keep changes staged
git reset --hard HEAD~1      # Undo commit, discard changes (dangerous)
git revert <commit>           # New commit that undoes a commit (safe for shared branches)
```

7.4 9.4 Branches and merging

```
git branch feature/login
git switch feature/login      # or: git checkout feature/login
git merge feature/login      # Merge into current branch
git merge --no-ff feature/login # Always create merge commit
```

7.4.1 Merge vs rebase

Merge	Rebase
Preserves history	Linear history
Creates merge commits	Rewrites commits
Safe for shared branches	Use only on local/feature branches

Golden rule: Never rebase commits that others have pulled.

7.5 9.5 Remote repositories

```
git remote -v
git fetch origin          # Download objects, don't merge
git pull origin main      # fetch + merge
git push origin main
git push -u origin feature/x # Set upstream
```

7.5.1 Tracking branches

`origin/main` is a **remote-tracking branch** — your local copy of what's on the server.

7.6 9.6 .gitignore and hooks

7.6.1 .gitignore

```
node_modules/
*.log
.env
dist/
.terraform/
```

7.6.2 Git hooks (`.git/hooks/`)

Scripts that run on events: `pre-commit`, `commit-msg`, `pre-push`. Used for linting, tests, and commit message format.

Example with **pre-commit** framework:

```
pip install pre-commit
# .pre-commit-config.yaml defines hooks
```

7.7 9.7 Chapter summary

- Git stores blobs, trees, commits, and tags; branches are movable pointers.
 - Use `reflog` to recover “lost” commits.
 - Prefer **merge** on shared branches; **rebase** to clean up local work.
 - Automate quality with hooks and `.gitignore`.
-

7.8 Lab 9.1

1. Create a repo with 5 commits on `main`.
 2. Create a feature branch, make 2 commits, merge with `--no-ff`.
 3. Use `git log --graph --oneline --all` to visualize the DAG.
 4. Reset hard to an earlier commit, then recover using `git reflog`.
-

Next: [Chapter 20 — IaC Principles](#)

8 Chapter 20: IaC Principles — State, Idempotency, and Drift

DevOps Handbook — Part VI, Pages 361–380

8.1 20.1 What is Infrastructure as Code?

Infrastructure as Code (IaC) means defining servers, networks, databases, and cloud resources in **machine-readable files** stored in version control—treated like application code.

Benefits:

- **Reproducibility** — Identical dev/staging/prod environments
 - **Auditability** — Git history shows who changed what
 - **Speed** — Provision in minutes, not days
 - **Documentation** — Code is the source of truth
 - **Disaster recovery** — Rebuild from code
-

8.2 20.2 Declarative vs imperative

Style	You specify	Example tools
Declarative	Desired end state	Terraform, CloudFormation, Kubernetes YAML
Imperative	Steps to execute	Ansible (mostly), shell scripts, AWS CLI scripts

Declarative tools compare desired state to actual state and compute a plan. **Imperative** tools run commands in sequence.

Most modern IaC is declarative with imperative escape hatches.

8.3 20.3 Idempotency

An operation is **idempotent** if running it multiple times produces the same result.

```
# Ansible – idempotent: "ensure nginx is installed and running"
- name: Install nginx
  apt:
    name: nginx
    state: present

- name: Start nginx
  service:
    name: nginx
    state: started
```

Running the playbook twice does not install nginx twice or restart unnecessarily.

Why it matters: CI/CD pipelines re-run constantly; non-idempotent scripts cause drift and outages.

8.4 20.4 State management

Tools like **Terraform** track what they created in a **state file** (`terraform.tfstate`).

State maps resource addresses to real cloud IDs:

```
{
  "resources": [{
    "type": "aws_s3_bucket",
    "name": "logs",
    "instances": [{
      "attributes": { "id": "my-app-logs-bucket", "arn": "arn:aws:s3:::..." }
    }]
  }]
}
```

8.4.1 State best practices

1. **Remote state** — Store in S3 + DynamoDB lock (AWS), GCS, Terraform Cloud
2. **Never commit secrets** in state (may contain sensitive values)
3. **Locking** — Prevent concurrent `terraform apply` from corrupting state
4. **State encryption** at rest

8.5 20.5 Configuration drift

Drift occurs when live infrastructure diverges from IaC definitions (manual console changes, emergency fixes).

Detection:

- `terraform plan` shows drift before apply
- AWS Config rules
- Scheduled drift detection jobs

Remediation:

- **Reconcile** — Apply IaC to restore desired state
- **Import** — Bring manual resources under IaC management
- **Prevent** — Restrict console access; require IaC PRs

8.6 20.6 IaC workflow in teams

Developer → PR with `.tf` changes → CI runs `terraform plan` → Review → Merge
→ CD runs `terraform apply` → Slack notification

8.6.1 Directory structure (example)

```
infrastructure/
├── modules/
│   ├── vpc/
│   ├── eks/
│   └── rds/
├── environments/
│   ├── dev/
│   ├── staging/
│   └── prod/
└── README.md
```

Each environment calls shared modules with different variables.

8.7 20.7 Testing IaC

Level	Tool	What it checks
Lint	<code>terraform fmt</code> , <code>tflint</code> , <code>cfn-lint</code>	Syntax and style
Static security	<code>checkov</code> , <code>tfsec</code> , <code>kics</code>	Misconfigurations
Plan	<code>terraform plan</code> in CI	Expected changes
Integration	Terratest, kitchen- terraform	Real deploy in test account

8.8 20.8 Chapter summary

- IaC makes infrastructure versioned, repeatable, and reviewable.
- Prefer **declarative**, **idempotent** tools.
- Manage **state** remotely with locking; detect and fix **drift**.

8.9 Lab 20.1

1. Manually change a tag on a Terraform-managed resource in the AWS console.
2. Run `terraform plan` and observe drift output.
3. Run `terraform apply` to reconcile.

Next: [Chapter 21 — Terraform](#)

9 Chapter 21: Terraform — HCL, Providers, and Modules

DevOps Handbook — Part VI, Pages 381–410

9.1 21.1 Terraform overview

Terraform (HashiCorp / OpenTofu fork) is the most widely used multi-cloud IaC tool. You write **HCL** (HashiCorp Configuration Language); Terraform calls cloud APIs via **providers**.

```
terraform init      # Download providers, init backend
terraform fmt       # Format code
terraform validate  # Syntax check
terraform plan      # Preview changes
terraform apply     # Create/update/destroy resources
terraform destroy   # Tear down (use carefully)
```

9.2 21.2 Basic configuration

`main.tf`:

```
terraform {
  required_version = ">= 1.5"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }

  backend "s3" {
    bucket     = "my-terraform-state"
    key        = "prod/vpc/terraform.tfstate"
    region     = "us-east-1"
    dynamodb_table = "terraform-locks"
    encrypt    = true
  }
}

provider "aws" {
  region = var.aws_region
}

resource "aws_s3_bucket" "logs" {
  bucket = "${var.project_name}-logs-${var.environment}"

  tags = {
    Project      = var.project_name
    Environment  = var.environment
    ManagedBy    = "terraform"
  }
}
```

`variables.tf`:

```
variable "aws_region" {
  type    = string
  default = "us-east-1"
}
```

```
variable "project_name" {
  type = string
}

variable "environment" {
  type = string
  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "environment must be dev, staging, or prod."
  }
}
```

outputs.tf:

```
output "logs_bucket_name" {
  value      = aws_s3_bucket.logs.id
  description = "Name of the logs S3 bucket"
}
```

Apply with: `terraform apply -var="project_name=myapp" -var="environment=dev"`

9.3 21.3 Resources, data sources, and dependencies

9.3.1 Data sources — read existing infrastructure

```
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
  }
}

resource "aws_instance" "web" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "t3.micro"
}
```

9.3.2 Implicit vs explicit dependencies

Terraform infers dependencies from references. Use `depends_on` when ordering matters but no attribute reference exists.

9.4 21.4 Modules — reusable infrastructure

modules/vpc/main.tf:

```
resource "aws_vpc" "this" {
  cidr_block      = var.cidr_block
  enable_dns_hostnames = true
  tags = { Name = var.name }
}

resource "aws_subnet" "public" {
  count            = length(var.public_subnet_cidrs)
  vpc_id          = aws_vpc.this.id
  cidr_block      = var.public_subnet_cidrs[count.index]
  availability_zone = var.availability_zones[count.index]
  map_public_ip_on_launch = true
}
```

Using the module:

```
module "vpc" {
  source = "../modules/vpc"

  name           = "prod-vpc"
  cidr_block     = "10.0.0.0/16"
  public_subnet_cidrs = ["10.0.1.0/24", "10.0.2.0/24"]
  availability_zones = ["us-east-1a", "us-east-1b"]
}
```

Publish modules to **Terraform Registry** or private registries for org-wide reuse.

9.5 21.5 Lifecycle meta-arguments

```
resource "aws_instance" "web" {
  ami           = var.ami_id
  instance_type = "t3.micro"

  lifecycle {
    create_before_destroy = true
    prevent_destroy       = false # Set true for prod databases
    ignore_changes        = [tags["LastReboot"]] # Ignore specific drift
  }
}
```

9.6 21.6 Workspaces and environments

```
terraform workspace list
terraform workspace new staging
terraform workspace select prod
```

Workspaces share the same backend key by default—often **separate directories per environment** (with separate state keys) is clearer for production.

9.7 21.7 CI/CD integration

GitHub Actions example (plan on PR):

```
- name: Terraform Plan
  run: |
    terraform init -input=false
    terraform plan -input=false -no-color -out=tfplan
  env:
    AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
    AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
```

Apply only from protected `main` branch after approval.

9.8 21.8 Chapter summary

- Terraform workflow: **init** → **plan** → **apply** with remote state.
- Use **variables**, **outputs**, and **modules** for reuse.
- Run `terraform plan` in CI on every PR; apply from trusted pipelines only.

9.9 Lab 21.1

1. Write Terraform to create an S3 bucket with versioning and encryption.
2. Extract a VPC module with 2 public subnets.
3. Configure S3 backend for remote state.
4. Run plan/apply in a free-tier AWS account.

Next: [Chapter 26 — Docker](#)

10 Chapter 26: Docker — Images, Dockerfile, and Multi-stage Builds

DevOps Handbook — Part VII, Pages 481–505

10.1 26.1 Containers in one paragraph

A **container** packages an application with its dependencies into an isolated process on a shared Linux kernel. Unlike VMs, containers share the host OS kernel—lightweight and fast to start.

Docker is the dominant tooling for building, shipping, and running containers locally and in CI.

10.2 26.2 Core concepts

Concept	Description
Image	Read-only template (layers)
Container	Running instance of an image
Dockerfile	Recipe to build an image
Registry	Storage for images (Docker Hub, ECR)
Volume	Persistent data outside container lifecycle
Network	Connect containers to each other and the host

10.3 26.3 Essential commands

```
docker pull nginx:1.25-alpine
docker images
docker run -d -p 8080:80 --name web nginx:1.25-alpine
docker ps
docker logs web
docker exec -it web sh
docker stop web && docker rm web
docker build -t myapp:1.0 .
docker tag myapp:1.0 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:1.0
docker push 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:1.0
```

10.4 26.4 Dockerfile best practices

```
# Use specific version tags, not :latest in production
FROM node:20-alpine AS builder

WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production=false

COPY . .
RUN npm run build
```

```
# Multi-stage: smaller final image
FROM node:20-alpine AS runtime

RUN addgroup -g 1001 -S app && adduser -u 1001 -S app -G app
WORKDIR /app
USER app

COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY package.json .

EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s CMD wget -qO- http://localhost:3000/health || exit 1

CMD ["node", "dist/server.js"]
```

10.4.1 Best practices checklist

- Use **multi-stage builds** to shrink images
- Run as **non-root** user
- Minimize layers; combine RUN commands where sensible
- Use `.dockerignore` (exclude `node_modules`, `.git`, etc.)
- Pin base image digests for supply chain security
- Add **HEALTHCHECK** for orchestrators

10.5 26.5 Docker Compose for local development

`docker-compose.yml` :

```
services:
  api:
    build: .
    ports:
      - "3000:3000"
    environment:
      DATABASE_URL: postgres://user:pass@db:5432/app
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: app
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user -d app"]
      interval: 5s
      timeout: 5s
      retries: 5

volumes:
  pgdata:
```

```
docker compose up -d
docker compose logs -f api
docker compose down -v # Remove volumes too
```

10.6 26.6 Security basics

- Scan images: `docker scout cves` or **Trivy**
 - Do not store secrets in images or Dockerfile `ENV`
 - Use read-only root filesystem where possible (`--read-only`)
 - Drop capabilities: `--cap-drop=ALL`
-

10.7 26.7 Chapter summary

- Images are built from Dockerfiles; containers are ephemeral runtime instances.
 - **Multi-stage builds** and non-root users are production essentials.
 - **Docker Compose** orchestrates multi-container local stacks.
-

10.8 Lab 26.1

1. Containerize a simple Node or Python web app.
 2. Add multi-stage build and health check.
 3. Run with Compose alongside PostgreSQL.
 4. Scan the image with Trivy.
-

Next: [Chapter 37 — CI/CD Pipeline Design](#)

11 Chapter 37: CI/CD Concepts and Pipeline Design

DevOps Handbook — Part IX, Pages 721–745

11.1 37.1 Continuous Integration (CI)

CI means developers merge code to a shared branch **frequently** (at least daily), and each merge triggers an **automated build and test** pipeline.

Goals:

- Detect integration bugs early
- Keep `main` always deployable
- Reduce merge hell from long-lived branches

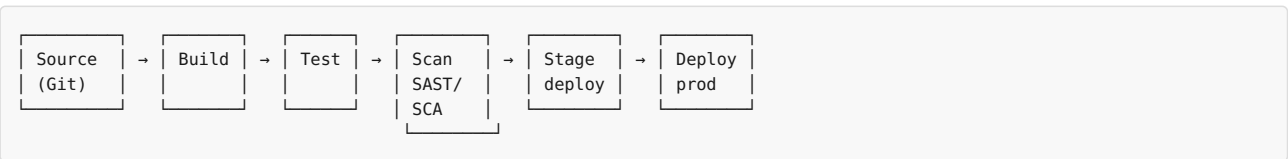
11.2 37.2 Continuous Delivery vs Continuous Deployment

Practice	Definition
Continuous Delivery	Every change is releasable; deploy to prod is a manual (or one-click) decision
Continuous Deployment	Every change that passes tests is automatically deployed to production

Most enterprises use **Continuous Delivery** with approval gates for production.

11.3 37.3 Pipeline stages

A typical pipeline:



Stage	Activities
Source	Webhook on push/PR
Build	Compile, docker build, artifact upload
Test	Unit, integration, contract tests
Scan	SAST, dependency scan, container scan, IaC scan
Stage deploy	Deploy to staging; smoke tests
Prod deploy	Blue/green, canary, or rolling; approval gate

11.4 37.4 GitHub Actions example

`.github/workflows/ci.yml`:

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Node
        uses: actions/setup-node@v4
        with:
          node-version: "20"
          cache: npm

      - run: npm ci
      - run: npm run lint
      - run: npm test -- --coverage

      - name: Build Docker image
        run: docker build -t myapp:${{ github.sha }} .

      - name: Scan image
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: myapp:${{ github.sha }}
          severity: CRITICAL,HIGH
          exit-code: 1

  deploy-staging:
    needs: build-and-test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to staging
        run: ./scripts/deploy.sh staging ${{ github.sha }}
```

11.5 37.5 Branching and trunk-based development

Trunk-based development (recommended for CI/CD):

- Short-lived feature branches (< 2 days)
- Frequent merges to `main`
- Feature flags hide incomplete work

Avoid long-running `develop` branches that diverge from production reality.

11.6 37.6 Deployment strategies

Strategy	How it works	Risk
Rolling	Replace instances gradually	Medium
Blue/green	Switch traffic between two identical envs	Low downtime
Canary	Route small % traffic to new version	Lowest blast radius

Recreate	Tear down old, start new	Downtime
-----------------	--------------------------	----------

11.7 37.7 Pipeline secrets and artifacts

- Store secrets in **GitHub Secrets**, **Vault**, or cloud secret managers—never in YAML.
- Pin action versions (`@v4` , commit SHA) for supply chain security.
- Upload build artifacts (JAR, Docker image digest) for downstream deploy jobs.
- Sign artifacts (Sigstore/cosign) for provenance.

11.8 37.8 DORA metrics in your pipeline

Track:

1. **Deployment frequency** — How often you deploy to prod
2. **Lead time for changes** — Commit to prod duration
3. **Change failure rate** — % of deploys causing incidents
4. **MTTR** — Time to restore after failure

Instrument pipelines to emit these metrics to your observability platform.

11.9 37.9 Chapter summary

- CI validates every merge; CD/CD automates release to staging/prod.
- Design pipelines with **build → test → scan → deploy** stages.
- Prefer trunk-based dev, pinned dependencies, and safe deployment strategies.

11.10 Lab 37.1

Build a full GitHub Actions pipeline for a sample app: lint, test, Docker build, Trivy scan, and deploy to staging on merge to `main`.

Continue: Chapter 38 — GitHub Actions Deep Dive (outline in TOC)